

KONZEPT

JAMA MCP Service — Konzept

Architektur, Anwendungsfälle, Tool-Katalog und Betrieb

Inhalt

1. Ausgangslage und Abgrenzung

1.1 Was es bereits gibt

1.2 Ziel dieses Services

2. Erkenntnisse aus der API-Dokumentation

2.1 Basis

2.2 Authentifizierung

2.3 Throttling — der wichtigste Constraint

2.4 Pagination

2.5 include-Parameter

2.6 PATCH-Semantik

2.7 Suche (GET /abstractitems)

2.8 Ressourcen-Überblick (relevant für den Tool-Schnitt)

3. Architektur

3.1 Komponenten

3.2 Technologie

3.3 Zwei getrennte Auth-Ebenen

3.4 Caching-Strategie

3.5 Antwort-Aufbereitung (Token-Ökonomie)

4. Use Cases

4.1 Requirements Engineering

4.2 Traceability und Impact-Analyse

4.3 Test-Management

4.4 Review und Freigabe

4.5 Baselines, Historie und Änderungsmanagement

4.6 Bulk-Operationen und Migration

4.7 Reporting, Compliance und Governance

5. Tool-Katalog

5.1 Toolset core — Discovery, Suche, Navigation (read)

5.2 Toolset trace — Traceability (read + Analyse)

5.3 Toolset write — Schreibende Item-Operationen

5.4 Toolset collab — Kommentare und Workflow

5.5 Toolset test — Test-Management

5.6 Toolset history — Baselines, Versionen, Aktivitäten

5.7 Toolset review — Reviews (Jama labs, ab 9.32)

5.8 Toolset files — Anhänge und Reports

5.9 MCP Resources

5.10 MCP Prompts

5.11 Was bewusst nicht als Tool umgesetzt wird

6. Admin-Dashboard

6.1 PIN-Authentifizierung

6.2 Seitenstruktur

6.3 Datenmodell (PostgreSQL, Drizzle)

7. Sicherheit

8. Testkonzept

9. Umsetzungsstand

Nicht umgesetzt

10. Entscheidungen

10.1 Getroffen (2026-08-12)

10.2 Noch offen

Quellen

1. Ausgangslage und Abgrenzung

1.1 Was es bereits gibt

Lösung	Umfang	Lücke für uns
Offizieller Jama MCP Server (ab Jama Connect 9.35, Mai 2026)	Breites Toolset, respektiert Jama-Permissions und Audit	Lizenz-/Versionsabhängig, keine eigene Key-Verwaltung, keine eigene Usage-Sicht, kein eigener Zuschnitt der Tools
t-j-thomas/jama-mcp-server	Python, read-only, Wrapper um py-jama-rest-client	Nur lesend, nur stdio, kein Multi-User, kein Rate-Limit-Management, kein Admin
jamasoftware-ps/REST-References	Code-Beispiele (Auth, Pagination, Suche, Update)	Nur Beispiele

Entscheidungspunkt für den Kunden: Wenn die Jama-Instanz auf 9.35+ läuft und der offizielle MCP-Server lizenziert ist, sollte man ihn zumindest evaluieren. Der Eigenbau rechtfertigt sich über: eigene Key-Ausgabe und -Rotation, Usage- und Kosten-Transparenz, projektbezogene Allowlists, harte Write-Guards und einen aufgabenorientierten Tool-Zuschnitt statt 1:1-Endpoint-Mapping.

1.2 Ziel dieses Services

- MCP-Server** — bringt Jama-Daten und -Operationen in Claude, Cursor, Copilot, VS Code.
- Zentrale Credential-Verwaltung** — Jama-Zugangsdaten liegen verschlüsselt im Service, nicht in `.env`-Dateien auf 20 Entwickler-Laptops.
- Nutzungs-Transparenz** — wer ruft welches Tool wie oft auf, mit welcher Latenz, welchen Fehlern und welchem geschätzten Token-Verbrauch.
- Governance** — Read-only-Kill-Switch, Projekt-Allowlist, Audit-Trail für jede schreibende Operation (relevant für ASPICE / ISO 26262 / IEC 62304 / FDA-Kontexte, in denen Jama typischerweise eingesetzt wird).

2. Erkenntnisse aus der API-Dokumentation

Quellen: rest.jamasoftware.com, dev.jamasoftware.com/api, [API Cookbook](#), [Throttle-Limits](#).

2.1 Basis

- **Base-Path:** <https://<instanz>.jamacloud.com/rest/v1/> — Versionen: [v1](#) (stabil), [latest](#), [labs](#) (experimentell, u. a. Reviews und Reports).
- **Swagger:** Jede Cloud-Instanz stellt unter [/api-docs/](#) eine Swagger-UI bereit, die jedoch eine Anmeldung erfordert ([/rest/v1/](#) antwortet unauthentifiziert mit [401](#), [/api-docs/](#) leitet auf die Login-Seite um). Für die exakte Verifikation von Feldnamen und Custom-Field-IDs wird deshalb ein Zugang zu einer Test- oder Sandbox-Instanz benötigt.

2.2 Authentifizierung

Verfahren	Details	Bewertung
OAuth 2.0 Client Credentials	POST /rest/oauth/token mit Basic-Auth aus Client-ID/Secret, grant_type=client_credentials . Antwort: access_token + expires_in (default 3600 s)	Primärweg. Funktioniert auch mit SSO. Secret wird nur einmal angezeigt
Basic Auth	Authorization: Basic base64(user:pass)	Fallback. Funktioniert nicht in SAML/SSO-Umgebungen

Zusatz-Header [x-jama-date-fields-with-time](#) liefert volle ISO-8601-Zeitstempel (ab 8.59).

Lizenz-Einschränkung: REST-Zugriff nur mit *Named Creator*-Lizenz (ab Jama 9.4). Creator-Float-Lizenzen haben keinen API-Zugriff. Das muss beim Credential-Setup geprüft werden.

2.3 Throttling — der wichtigste Constraint

10 Requests/Sekunde **pro Environment** und ein Queue-Limit von 100 Requests. Bei Überschreitung: HTTP [429](#).

Das Limit gilt **instanzweit**, nicht pro Benutzer. Ein MCP-Server, der naiv parallelisiert, legt die Jama-Instanz für alle anderen Integrationen lahm. Daraus folgen zwingend:

- Globaler Token-Bucket **pro Jama-Verbindung** (nicht pro API-Key), konfigurierbar, Default konservativ bei ca. 6–8 req/s, um Luft für andere Integrationen zu lassen.
- Retry mit exponentiellem Backoff nach Jama-Empfehlung: 3 s, 5 s, 8 s, 13 s (Fibonacci), mit Jitter, max. 4 Versuche.
- Aggressives Caching von Stammdaten (siehe 3.4).
- Live-Anzeige der Rate-Limit-Auslastung im Admin-Dashboard.

2.4 Pagination

- Query: `startAt` , `maxResults` — **Default 20, Maximum 50.**
- Response: `meta.pageInfo` mit `startIndex` , `resultCount` , `totalResults` .
- Konsequenz: Ein Projekt mit 5.000 Items braucht 100 Requests = ~15 s allein wegen des Rate-Limits. Tools müssen deshalb *immer* paginierte, kompakte Ergebnisse liefern und niemals implizit die ganze Menge laden.

2.5 `include` -Parameter

`?include=data.fields.priority&include=data.project` — liefert einen `linked` -Block mit den referenzierten Objekten. Spart Folge-Requests (Picklist-Werte, Projekt-Namen, User-Namen auflösen) und ist damit unser wichtigster Hebel gegen das Rate-Limit.

2.6 PATCH-Semantik

```
[{"op": "replace", "path": "/fields/name", "value": "Neuer Wert" }]
```

Operationen: `add` , `replace` , `remove` . Custom-Felder tragen ein `$` -Suffix mit der ItemType-ID: `/fields/customField$12` . **Wichtig:** Feldnamen sind pro ItemType unterschiedlich — deshalb braucht das LLM zwingend ein Schema-Discovery-Tool (siehe 4.1), bevor es schreibt. `PATCH /items` (Batch) läuft asynchron.

2.7 Suche (`GET /abstractitems`)

Parameter: `project` , `itemType` , `release` , `documentKey` , `contains` , `createdDate` , `modifiedDate` , `lastActivityDate` , `sortBy=feld.asc|desc` . `contains` unterstützt Lucene-Syntax inkl. Feldbezug und Wildcards: `contains=name:fire` , `contains=name:*pha` . Mehrfach gesetzte gleiche Parameter werden als **ODER** verknüpft, unterschiedliche als **UND**.

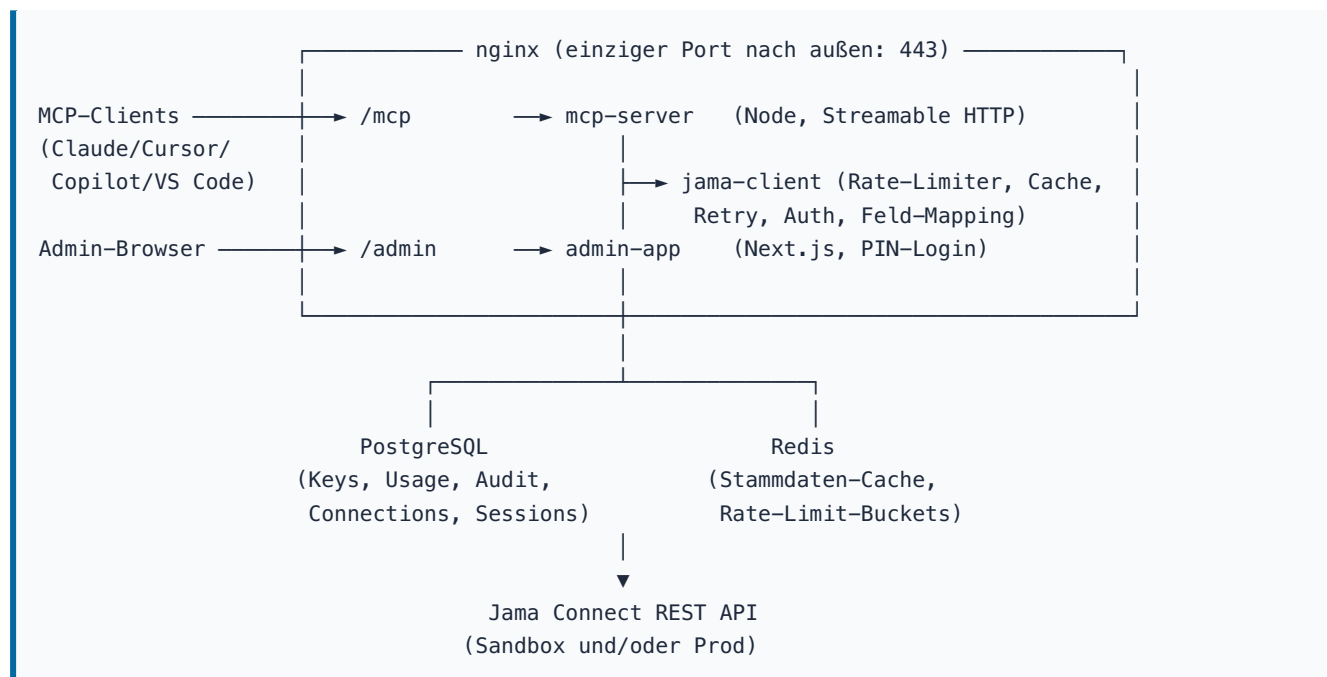
2.8 Ressourcen-Überblick (relevant für den Tool-Schnitt)

`abstractitems` , `activities` , `attachments` , `baselines` , `categories` , `comments` , `files` , `filters` , `items` (~47 Endpoints), `itemtypes` , `picklists` , `picklistoptions` , `projects` , `relationships` , `relationshiptypes` , `relationshiprulesets` , `releases` , `reviews` (labs), `reports` (labs), `system` , `tags` , `testplans` , `testcycles` , `testruns` , `users` , `usergroups` .

Insgesamt weit über 150 Endpoints. **Ein 1:1-Mapping auf MCP-Tools wäre ein Fehler** — das Tool-Listing allein würde das Kontextfenster fluten und die Trefferquote der Tool-Auswahl massiv verschlechtern. Wir bauen aufgabenorientierte Tools (siehe Kapitel 4).

3. Architektur

3.1 Komponenten



Nur nginx wird via `ports:` gemappt. `mcp-server` und `admin-app` nutzen `expose:`, Postgres und Redis gar kein Port-Mapping — Kommunikation ausschließlich über das Docker-Netzwerk. Persistente Daten (`pgdata`, `redisdata`) in Named Volumes.

3.2 Technologie

Baustein	Wahl	Begründung
Sprache	TypeScript durchgängig	Ein Stack für MCP-Server und Admin-UI, geteilte Typen und Zod-Schemas zwischen beiden. Vermeidet die Python/TS-Doppelwelt
MCP	@modelcontextprotocol/sdk	Referenz-SDK, unterstützt Tools, Resources und Prompts
Transport	Streamable HTTP (primär) + stdio (optional)	HTTP für den zentralen Multi-User-Betrieb inkl. Key-Auth und Usage-Logging. stdio als schlanker lokaler Modus für Entwickler
HTTP-Server	Fastify	Klein, schnell, gute Zod-Integration
Admin-UI	Next.js (App Router) + Tailwind + lucide-react	Server Components, Route Handlers als Admin-API. Icons ausschließlich lucide
DB	PostgreSQL	Vorgabe. Drizzle ORM + Drizzle-Migrations (Migrations laufen beim Deploy mit)

Baustein	Wahl	Begründung
Cache / Buckets	Redis	Rate-Limit-Bucket muss prozessübergreifend geteilt werden
Charts	Recharts	Für die Usage-Zeitreihen im Dashboard
Tests	Vitest + Testcontainers (Postgres) + nock /MSW für Jama-Mocks	MCDC-Matrix für Auth, Rate-Limiter und Write-Guards

3.3 Zwei getrennte Auth-Ebenen

Das ist der Kern des Designs und muss sauber getrennt bleiben:

Ebene 1 — MCP-Client zum Service: `Authorization: Bearer jama_mcp_<prefix>_<secret>`. Der Key wird bei Anlage **einmal** angezeigt, gespeichert wird nur `sha256(secret)` plus ein Prefix zur Identifikation (gleiche Mechanik wie Jamas OAuth-Secret).

Ebene 2 — Service zu Jama: Jeder API-Key ist an ein **Jama-Credential-Set** gebunden (OAuth Client-ID/Secret, verschlüsselt mit AES-256-GCM über `ENCRYPTION_KEY`).

Entscheidung: beide Modelle werden unterstützt, mit klarem Standard.

Standard — ein Jama-Credential pro Person. Gründe:

- Jamas eigene Berechtigungen greifen dann pro Benutzer — der MCP-Server wird nie zum Rechte-Eskalationspfad.
- Jamas Audit-Trail zeigt den echten Verursacher, nicht "MCP-Bot".
- Nutzung ist im Dashboard personenbezogen zuordenbar.

Zusätzlich — geteilte Service-Accounts für CI und Automatisierung. Diese werden im Datenmodell über `api_keys.account_type = 'service'` unterschieden und im Admin sichtbar als solche markiert. Für sie gelten strengere Vorgaben, die die UI erzwingt: Projekt-Allowlist ist Pflichtfeld (nicht leer), ein Ablaufdatum ist Pflicht, und ein benannter verantwortlicher Inhaber muss hinterlegt sein. In Usage und Audit erscheinen sie mit eigenem Badge, damit maschinelle Zugriffe nicht mit menschlichen vermischt werden.

3.4 Caching-Strategie

Datenart	TTL	Begründung
<code>projects</code> , <code>itemtypes</code> , <code>picklists</code> , <code>picklistoptions</code> , <code>relationshipatypes</code> , <code>usergroups</code>	6 h	Stammdaten, ändern sich selten, werden bei fast jedem Call gebraucht
<code>users</code> , <code>tags</code> , <code>releases</code>	1 h	Mittlere Änderungsrate
OAuth-Access-Token	<code>expires_in</code> – 60 s	Vermeidet unnötige Token-Requests
Item-Inhalte	kein Cache	Korrektheit geht vor; Requirements müssen aktuell sein

Cache-Invalidierung manuell aus dem Admin auslösbar. Cache-Hit-Rate wird im Dashboard ausgewiesen, weil sie direkt das Rate-Limit-Budget schont.

3.5 Antwort-Aufbereitung (Token-Ökonomie)

Der größte Unterschied zwischen einem API-Wrapper und einem guten MCP-Server. Rohe Jama-Item-Responses enthalten HTML-Descriptions, die einzeln mehrere tausend Token umfassen können. Regeln:

1. **Feld-Whitelist statt Vollausgabe** — Listen liefern nur `id`, `documentKey`, `name`, `itemType`, `status`, `modifiedDate`. Details erst per `jama_get_item`.
 2. **HTML zu Markdown** — Description-Felder werden konvertiert (Tabellen bleiben erhalten), `<img>` wird zu einem Platzhalter mit Attachment-Referenz.
 3. **Harte Truncation** — pro Antwort ein konfigurierbares Budget (Default ~15.000 Token). Bei Überschreitung: kürzen mit expliziter Notiz plus Hinweis, wie gezielter abzufragen ist.
 4. **Cursor-Pagination nach außen** — der MCP-Client bekommt einen opaken `cursor`, der intern auf `startAt` gemappt wird. Kein `maxResults: 5000` möglich.
 5. **Picklist-IDs auflösen** — statt `"status": 307` liefert die Antwort `"status": "Approved"`. Ohne das muss das LLM raten oder Extra-Calls machen.
-

4. Use Cases

Gesammelt entlang der Rollen, die typischerweise mit Jama arbeiten. Sie sind die Grundlage für den Tool-Schnitt in Kapitel 5 — jedes Tool muss mindestens einen Use Case bedienen, und jeder Use Case muss durch Tools abgedeckt sein.

4.1 Requirements Engineering

#	Use Case
UC-01	"Zeig mir alle Anforderungen in Projekt X, die seit letzter Woche geändert wurden"
UC-02	"Finde alle Requirements, die 'Bremsdruck' im Namen oder in der Beschreibung enthalten"
UC-03	"Lege unter Ordner Y eine neue Systemanforderung mit diesem Text an"
UC-04	"Welche Requirements haben eine leere Beschreibung oder fehlende Pflichtfelder?"
UC-05	"Formuliere diese Anforderung nach EARS/INCOSE-Regeln um und schreibe sie zurück"
UC-06	"Zerlege diesen Lastenheft-Absatz in einzelne, atomare Anforderungen und lege sie an"
UC-07	"Welche Items stehen seit über 30 Tagen auf Status 'Draft'?"
UC-08	"Finde inhaltlich doppelte oder widersprüchliche Anforderungen in diesem Modul"
UC-09	"Prüfe alle Requirements gegen unsere Namenskonvention und liste Verstöße"
UC-10	"Zeig mir die Struktur (Baum) von Projekt X bis Ebene 3"

4.2 Traceability und Impact-Analyse

#	Use Case
UC-11	"Zeig die komplette Kette von Stakeholder-Requirement REQ-12 bis zum Testfall"
UC-12	"Welche Requirements in Projekt X haben keine Downstream-Verknüpfung? (Trace-Gap)"
UC-13	"Impact-Analyse: Was hängt alles an REQ-123, wenn ich es ändere?"
UC-14	"Verknüpfe REQ-45 mit TC-88 als 'verified by'"
UC-15	"Erzeuge eine Traceability-Matrix Requirements zu Testfällen als Tabelle"
UC-16	"Welche Beziehungen sind nach der letzten Änderung 'suspect' und müssen geprüft werden?"
UC-17	"Verstößt diese geplante Verknüpfung gegen die Relationship-Rules des Projekts?"
UC-18	"Welche Safety-Requirements haben keine Verifikation? (Compliance-Nachweis)"

4.3 Test-Management

#	Use Case
UC-19	"Erstelle einen Testplan mit Testgruppen aus den Requirements von Feature X"
UC-20	"Generiere Testschritte aus dieser Anforderung und lege den Testfall an"
UC-21	"Welche Testruns sind im aktuellen Testzyklus noch offen?"
UC-22	"Setze Testrun 42 auf PASSED mit diesem Kommentar"
UC-23	"Testabdeckung: Welche Requirements haben keinen zugeordneten Testfall?"
UC-24	"Fasse den letzten Testzyklus zusammen: Pass/Fail/Blocked pro Testgruppe"
UC-25	"Welche Tests sind zuletzt fehlgeschlagen, und was steht in den Kommentaren?"

4.4 Review und Freigabe

#	Use Case
UC-26	"Welche Reviews laufen gerade, und wer hat noch nicht abgestimmt?"
UC-27	"Fasse die Kommentare aus Review 17 zusammen und gruppiere sie nach Thema"
UC-28	"Zeig alle offenen Kommentare zu meinem Modul"
UC-29	"Antworte auf Kommentar 991 mit dieser Begründung"
UC-30	"Führe die Workflow-Transition 'Approve' auf Item Y aus"
UC-31	"Welche Workflow-Übergänge sind für Item Y überhaupt erlaubt?"

4.5 Baselines, Historie und Änderungsmanagement

#	Use Case
UC-32	"Was hat sich zwischen Baseline v1.0 und v1.1 geändert?"
UC-33	"Erstelle eine Baseline für den aktuellen Stand von Modul X"
UC-34	"Versionshistorie von REQ-123: Wer hat wann was geändert?"
UC-35	"Zeig alle Änderungen der letzten 24 Stunden in Projekt X"
UC-36	"Wer hat Item Z gelöscht, und kann man es wiederherstellen?"
UC-37	"Erzeuge Release Notes aus den Items von Release 2026.1"

4.6 Bulk-Operationen und Migration

#	Use Case
UC-38	"Lege aus dieser CSV-Liste 40 Requirements im Ordner Y an"
UC-39	"Setze bei allen Items mit Tag 'Legacy' die Priorität auf Low"
UC-40	"Verschiebe Ordner X unter Y"
UC-41	"Dupliziere Modul X als Basis für Variante B"
UC-42	"Hänge dieses PDF an REQ-77 an"
UC-43	"Tagge alle Items aus diesem Suchergebnis mit 'Sprint-24'"

4.7 Reporting, Compliance und Governance

#	Use Case
UC-44	"Wie viele Requirements pro Status in Projekt X? (Kennzahlen)"
UC-45	"Fortschritt Release 2026.1: Wie viele Items sind abgeschlossen?"
UC-46	"Generiere ein Lastenheft-Kapitel aus Ordner X als Markdown"
UC-47	"Führe den gespeicherten Jama-Filter 'Offene CRs' aus"
UC-48	"Starte den Jama-Report Y und gib mir den Download-Link"
UC-49	"Audit-Trail für Item X exportieren"
UC-50	"Welche Admin-Aktivitäten gab es im letzten Monat?"
UC-51	"Welche Projekte sehe ich, und welche Rechte habe ich dort?"

5. Tool-Katalog

Leitprinzip: aufgabenorientiert, nicht endpoint-orientiert. 53 Tools statt 150+, organisiert in **Toolsets**, die pro API-Key einzeln freischaltbar sind. Ein Key, der nur **core** und **trace** hat, sieht die Write-Tools gar nicht — das reduziert Kontext-Last und Fehlbedienungs-Risiko zugleich.

5.1 Toolset **core** — Discovery, Suche, Navigation (read)

Tool	Endpoints	Deckt ab
jama_whoami	GET /users/current , GET /rest	Verbindungstest, Lizenz-/Rechteprüfung, API-Version
jama_list_projects	GET /projects	UC-51
jama_get_project_schema	GET /itemtypes , /picklists , /picklistoptions , /relationshipstypes	Pflicht-Vorstufe vor jedem Schreibvorgang. Liefert erlaubte ItemTypes, Feldnamen inkl. customField\$NN , Picklist-Werte
jama_search_items	GET /abstractitems	UC-01, UC-02, UC-07 — Lucene- contains , Datumsfilter, sortBy , Cursor
jama_get_item	GET /items/{id} + include	Volldetail eines Items, HTML→Markdown
jama_get_items_batch	mehrfach GET /items/{id}	Token-effizient mehrere Items in einem Call
jama_browse_tree	GET /items/{id}/children , /parent , /location	UC-10
jama_run_filter	GET /filters , /filters/{id}/results , /count	UC-47 — nutzt in der Jama-UI gepflegte Filter, sehr mächtig
jama_list_releases	GET /releases	UC-37, UC-45
jama_list_tags	GET /tags , GET /tags/{id}/items	UC-43
jama_list_users	GET /users , GET /usergroups	Zuweisungen auflösen

5.2 Toolset **trace** — Traceability (read + Analyse)

Tool	Endpoints	Deckt ab
jama_get_relationships	GET /items/{id}/upstream-/downstreamrelationships	UC-13
jama_trace_chain	rekursiv über up-/downstream	UC-11 — Komposit-Tool , ersetzt 10–30 Einzel-Calls. Tiefe und Richtung parametrisiert, Zyklenerkennung

Tool	Endpoints	Deckt ab
jama_trace_matrix	GET /relationships mit Projekt-Scope	UC-15 — kompakte Matrix statt Objektgraph
jama_find_trace_gaps	Suche + Relationship-Abgleich	UC-12, UC-18, UC-23 — Komposit , das eigentliche Compliance-Werkzeug
jama_check_relationship_rules	GET /relationshiprulesets	UC-17 — Trockenlauf vor dem Verknüpfen

5.3 Toolset **write** — Schreibende Item-Operationen

Nur mit expliziter Freischaltung, unterliegt dem globalen Read-only-Kill-Switch, der Projekt-Allowlist und vollständigem Audit-Logging.

Tool	Endpoints	Deckt ab
jama_create_item	POST /items	UC-03, UC-06, UC-20
jama_update_item	PATCH /items/{id}	UC-05
jama_bulk_update_items	PATCH /items (async)	UC-39 — mit Vorschau-Modus (dryRun)
jama_bulk_create_items	mehrfach POST /items	UC-38 — sequenziell mit Rate-Limit-Respekt, Teilerfolg-Report
jama_delete_item	DELETE /items/{id}	Erfordert confirm: true , immer Audit-Eintrag
jama_move_item	PUT /items/treeLocation	UC-40
jama_duplicate_item	POST /items/{id}/duplicate	UC-41
jama_create_relationship	POST /relationships	UC-14
jama_delete_relationship	DELETE /relationships/{id}	Mit confirm: true
jama_manage_tags	POST / DELETE /items/{id}/tags	UC-43
jama_lock_item	GET / PUT /items/{id}/lock	Sperrung prüfen und setzen vor Bearbeitung

5.4 Toolset **collab** — Kommentare und Workflow

Tool	Endpoints	Deckt ab
jama_list_comments	GET /items/{id}/comments , /comments/{id}/replies	UC-28
jama_add_comment	POST /comments	UC-29

Tool	Endpoints	Deckt ab
jama_get_workflow_options	GET /items/{id}/workflowtransitionoptions	UC-31
jama_execute_workflow_transition	POST /items/{id}/workflowtransitions	UC-30 — Write-Guard + Audit

5.5 Toolset **test** — Test-Management

Tool	Endpoints	Deckt ab
jama_list_testplans	GET /testplans	UC-19
jama_create_testplan	POST /testplans , /testgroups , /testgroups/{id}/testcases	UC-19 — Komposit über drei Endpoints
jama_list_testcycles	GET /testcycles	UC-21
jama_create_testcycle	POST /testplans/{id}/testcycles	UC-19
jama_list_testruns	GET /testcycles/{id}/testruns , GET /testruns	UC-21, UC-25
jama_update_testrun	PUT / PATCH /testruns	UC-22 — inkl. Steps und Custom Fields
jama_testcycle_summary	GET /testruns + Aggregation	UC-24 — Komposit, liefert Kennzahlen statt Rohdaten

5.6 Toolset **history** — Baselines, Versionen, Aktivitäten

Tool	Endpoints	Deckt ab
jama_list_baselines	GET /baselines	UC-32
jama_create_baseline	POST /baselines (labs)	UC-33
jama_compare_baselines	GET /baselines/{id}/versioneditems (2x) + Diff	UC-32 — Komposit , feldweiser Diff mit added/removed/changed
jama_get_item_history	GET /items/{id}/versions , /versions/{n}/versioneditem	UC-34, UC-49
jama_get_activities	GET /activities , /activities/adminActivity	UC-35, UC-50
jama_restore_deleted	POST /activities/{id}/restore	UC-36 — <code>confirm: true</code> + Audit

5.7 Toolset `review` — Reviews (Jama `labs` , ab 9.32)

Tool	Endpoints	Deckt ab
<code>jama_list_reviews</code>	<code>GET /reviews</code>	UC-26
<code>jama_get_review_status</code>	<code>/reviews/{id}/revisions/{rid}/progress</code> , <code>/participants</code>	UC-26
<code>jama_list_review_comments</code>	<code>GET /reviews/{id}/comments</code>	UC-27

Als `labs` -Endpoints ohne Support-Zusage — Toolset wird nur aktiviert, wenn die Ziel-Instanz die Version mitbringt (Feature-Detection über `GET /rest` beim Verbindungstest).

5.8 Toolset `files` — Anhänge und Reports

Tool	Endpoints	Deckt ab
<code>jama_list_attachments</code>	<code>GET /items/{id}/attachments</code>	UC-42
<code>jama_upload_attachment</code>	<code>POST /projects/{id}/attachments</code> , <code>PUT /attachments/{id}/file</code> , <code>POST /items/{id}/attachments</code>	UC-42 — Komposit über drei Schritte
<code>jama_download_attachment</code>	<code>GET /attachments/{id}/file</code>	Textextraktion für PDF/DOCX optional
<code>jama_list_reports</code> / <code>jama_run_report</code>	<code>GET /reports/metadata</code> , <code>POST /reports/{id}</code> (labs, async)	UC-48

5.9 MCP Resources

Kontext, den der Client ohne Tool-Call laden kann:

- `jama://projects` — Projektliste
- `jama://project/{id}/schema` — ItemTypes, Felder, Picklists, Relationship-Typen
- `jama://item/{documentKey}` — einzelnes Item als Markdown
- `jama://server/capabilities` — aktive Toolsets, Limits, Instanz (Sandbox oder Prod)

5.10 MCP Prompts

Vorgefertigte Arbeitsabläufe, die den Nutzer durch mehrstufige Aufgaben führen:

- `jama_review_requirements` — Qualitätsprüfung eines Moduls (UC-04, UC-05, UC-08, UC-09)
- `jama_trace_gap_analysis` — Lückenanalyse mit Compliance-Bericht (UC-12, UC-18)
- `jama_testplan_from_requirements` — Testplan aus Requirements ableiten (UC-19, UC-20)
- `jama_baseline_diff_report` — Änderungsbericht zwischen zwei Baselines (UC-32)
- `jama_spec_to_items` — Freitext-Spezifikation in atomare Items zerlegen (UC-06)

5.11 Was bewusst nicht als Tool umgesetzt wird

`categories` , `synceditems` , `relationshiprulesets` (schreibend), `projects/{id}/permissions` (schreibend), `usergroups` (schreibend). Begründung: geringe Relevanz für LLM-gestützte Arbeit, hohes Schadenspotenzial bei Fehlbedienung, und jedes zusätzliche Tool verschlechtert die Tool-Auswahlqualität. Nachrüstbar, wenn ein konkreter Bedarf entsteht.

6. Admin-Dashboard

6.1 PIN-Authentifizierung

Der PIN ist bequem, aber schwach — deshalb wird er abgesichert:

- PIN mindestens 8 Stellen, gespeichert als **Argon2id**-Hash (nie im Klartext, nie in `.env` im Klartext bei laufendem Betrieb — Erst-PIN wird beim ersten Start gesetzt).
- **Rate-Limiting und Lockout:** 5 Fehlversuche pro IP in 15 Minuten, danach 15 Minuten Sperre mit exponentieller Verlängerung. Fehlversuche im Audit-Log.
- **Session-Cookie:** `httpOnly`, `secure`, `sameSite=strict`, TTL 8 Stunden, serverseitig widerrufbar.
- CSRF-Token für alle mutierenden Admin-Requests.
- Konstante Vergleichszeit gegen Timing-Angriffe.

Bezug zum Betriebsmodell: Der Service läuft zunächst nur lokal per docker-compose ohne Internet-Exposition — die Bedrohungslage für den PIN ist damit gering. Die oben genannten Maßnahmen werden trotzdem von Anfang an umgesetzt, weil ein späteres Deployment auf einen Server sonst genau an dieser Stelle nachgezogen werden müsste. Sobald der Service extern erreichbar wird, kommt verpflichtend ein zweiter Faktor (TOTP) oder eine IP-Allowlist im nginx hinzu; läuft der Service hinter einem Identity-Aware-Proxy mit SSO, übernimmt dessen Identität die Authentifizierung und der PIN bleibt nur als lokaler Fallback.

6.2 Seitenstruktur

1 — Übersicht

- Health-Kacheln: MCP-Server, Jama-Verbindung(en), Postgres, Redis
- Requests heute / 7 Tage / 30 Tage mit Sparkline
- **Rate-Limit-Auslastung gegen Jama als Live-Gauge** (aktuelle req/s gegen 10er-Budget) — die wichtigste operative Kennzahl
- Fehlerquote und p95-Latenz
- Aktive Keys, Top-5-Tools, letzte 10 Fehler
- Deutlicher Warn-Badge, wenn eine Verbindung auf **Produktion** zeigt

2 — API-Keys

- Tabelle: Name, Inhaber, Verbindung, Toolsets, Read-only-Flag, erstellt, zuletzt genutzt, Requests (30 Tage), Ablauf, Status
- Anlegen: Name, Inhaber, Jama-Verbindung, Jama-Credentials (OAuth-Client-ID/Secret oder Basic), Toolset-Auswahl, **Projekt-Allowlist**, Ablaufdatum, eigenes Rate-Limit, Read-only ja/nein
- Key wird **einmal** im Klartext angezeigt, mit fertigem Copy-Snippet für `claude_desktop_config.json` bzw. `.mcp.json`
- Aktionen: Verbindung testen (`GET /users/current`), rotieren, deaktivieren, löschen
- Ungenutzte Keys (> 60 Tage) werden markiert

3 — Jama-Verbindungen

- Mehrere Instanzen parallel (Sandbox, Produktion), jeweils Base-URL, Auth-Verfahren, verschlüsselte Credentials, erkannte API-Version, verfügbare [labs](#) -Endpoints
- Health-Check-Button, letzter Prüfzeitpunkt
- Produktions-Verbindungen sind rot markiert und erfordern eine zusätzliche Bestätigung beim Zuordnen zu einem Write-Key

4 — Nutzung / Analytics

- Zeitreihe der Aufrufe, filterbar nach Key, Tool, Toolset, Projekt, Zeitraum
- Top-Tools, Top-Keys, Top-Projekte
- Latenzverteilung p50/p95/p99, Fehler nach Statuscode (401/403/429/500)
- **Verhältnis MCP-Tool-Calls zu tatsächlichen Jama-API-Calls** und Cache-Hit-Rate
- **Geschätzter Token-Verbrauch je Tool** (Response-Bytes → Token-Schätzung) — macht sichtbar, welche Tools die LLM-Kosten treiben, und wo sich Antwort-Trimmung lohnt
- CSV-Export

5 — Audit-Log

- Jede schreibende Operation: Zeitpunkt, API-Key, Jama-Benutzer, Tool, Ziel-Item (Doc-Key), Ergebnis, Dauer
- Alle Admin-Aktionen: Key angelegt/rotiert/gelöscht, Einstellungen geändert, Login-Versuche
- Filterbar und als CSV exportierbar (Nachweisführung für Audits)
- Unveränderlich, nur Append; Aufbewahrung konfigurierbar

6 — Request-Explorer (Live-Log)

- Die letzten N Requests mit Tool, Parametern (Secrets redigiert), Dauer, Status
- Drilldown: welche Jama-Calls hat der Tool-Call ausgelöst, Cache-Treffer, Retries
- Wichtigstes Debug-Werkzeug im Betrieb

7 — Tool-Katalog

- Alle Tools mit Beschreibung, Eingabe-Schema, Toolset-Zugehörigkeit, Aufrufzahlen
- Tools global aktivieren/deaktivieren
- **"Ausprobieren"** — Tool direkt aus dem Admin gegen eine gewählte Verbindung ausführen und die exakte Antwort sehen, die ein LLM bekäme (inkl. Token-Schätzung)

8 — Einstellungen

- Globales Rate-Limit pro Verbindung, Retry-Policy, Cache-TTLs, Cache leeren
- Antwort-Budget und Truncation-Grenzen
- **Globaler Read-only-Kill-Switch** — sperrt sofort alle schreibenden Tools
- Toolset-Definitionen bearbeiten
- PIN ändern, Sessions aller Geräte beenden
- Aufbewahrungsfristen für Usage- und Audit-Daten

6.3 Datenmodell (PostgreSQL, Drizzle)

jama_connections	id, name, base_url, auth_type(oauth basic), credentials_enc, is_production, api_version, capabilities_json, last_health_check_at, health_status, created_at
api_keys	id, name, owner, account_type(user service), key_hash, key_prefix, connection_id, jama_credentials_enc, toolsets[], allowed_project_ids[], read_only, rate_limit_rps, expires_at, disabled_at, last_used_at, created_at, created_by
usage_events	id, ts, api_key_id, tool_name, toolset, project_id, duration_ms, status, error_code, jama_call_count, cache_hits, response_bytes, est_tokens
audit_log	id, ts, actor_type(api_key admin), actor_id, action, target_type, target_key, payload_redacted, result, ip
admin_sessions	id, token_hash, created_at, expires_at, ip, user_agent
login_attempts	id, ip, ts, success -- Basis für Lockout
settings	key, value_json, updated_at, updated_by

`usage_events` wird die größte Tabelle. Partitionierung nach Monat und ein Aufbewahrungs-Job (Default 180 Tage, im Admin einstellbar); für die Dashboard-Kacheln zusätzlich stündliche Aggregat-Tabellen, damit Charts nicht über Rohdaten aggregieren.

7. Sicherheit

Risiko	Maßnahme
Jama-Credentials im Klartext	AES-256-GCM mit <code>ENCRYPTION_KEY</code> , Key nur aus Umgebung, nie geloggt, nie über die Admin-API zurückgegeben
Gestohlener MCP-API-Key	Nur <code>sha256</code> -Hash gespeichert, Ablaufdatum, Rotation, sofortige Deaktivierung, Projekt-Allowlist begrenzt den Schaden
Rechte-Eskalation über den Service	Jama-Credentials pro Person statt Service-Account; Jamas eigene Berechtigungen bleiben wirksam
Ungewollte Schreibvorgänge durch das LLM	Write-Toolset opt-in, globaler Read-only-Kill-Switch, <code>confirm: true</code> bei destruktiven Tools, <code>dryRun</code> bei Bulk-Operationen, Audit-Trail
Prompt Injection über Jama-Inhalte	Requirements-Texte sind Fremdinhalte. Antworten werden als Daten markiert und ausgeliefert, eingebettete Instruktionen nicht als Anweisungen behandelt; Write-Tools erfordern immer eine Nutzer-Bestätigung im Client
Brute-Force auf den Admin-PIN	Argon2id, IP-Lockout mit exponentiellem Backoff, Audit-Eintrag, optional TOTP oder IP-Allowlist
Datenabfluss über Logs	Strukturierte Logs ohne Secrets und ohne Item-Inhalte; im Request-Explorer sind Parameter redigiert
Ausbremsen der Jama-Instanz	Konservativer globaler Token-Bucket, Backoff, Cache, Live-Gauge im Dashboard
Verwechslung Sandbox/Produktion	Produktions-Verbindungen visuell markiert, zusätzliche Bestätigung bei Write-Keys
Exponierte Dienste	Nur nginx nach außen; Security-Header (HSTS, CSP, nosniff, frame-ancestors); CORS als Allowlist

Eine Sicherheitsdurchsicht ist vor jedem Merge in `main` verbindlich, da sowohl Authentifizierung als auch Credential-Verwaltung betroffen sind.

8. Testkonzept

- **Unit** — Feld-Mapping, HTML→Markdown, Picklist-Auflösung, Truncation, Cursor-Kodierung.
 - **MCDC-Matrix** für die kritischen Entscheidungspfade:
 - Key-Auth: gültig × abgelaufen × deaktiviert × unbekannt × falsches Toolset × Projekt außerhalb Allowlist × globaler Read-only-Modus
 - Rate-Limiter: unter Limit × an Limit × 429-Antwort × Retry-Erschöpfung
 - OAuth: Token gültig × abgelaufen × Refresh fehlgeschlagen × Basic-Fallback
 - **Integration** — Jama-Client gegen [nock](#) -Fixtures, Admin-API gegen Testcontainers-Postgres.
 - **Contract-Tests gegen die Sandbox** — separater, manuell auslösbarer Lauf, der pro read-Tool einen echten Call macht und Schema-Abweichungen meldet (schützt vor stillen API-Änderungen bei Jama-Releases).
 - **Security-Tests je Route:** unauthentifiziert → 401, fremder Key → 403, Injection-Payloads in Suchparametern, Ownership-Prüfung im Admin.
-

9. Umsetzungsstand

Alles Folgende ist in v1.0.0 enthalten.

Bereich	Umgesetzt	Fundstelle
Fundament	Repo, docker-compose (nginx, Postgres), Drizzle-Schema und Migrationen, Secret-Erzeugung	<code>docker-compose.yml</code> , <code>src/db/</code>
Jama-Client	OAuth und Basic Auth, Token-Zwischenspeicher, Token-Bucket, Fibonacci-Backoff, Pagination, <code>include</code> , Fehlerübersetzung, Stammdaten-Cache, HTML→Markdown	<code>src/jama/</code>
MCP-Grundgerüst	Streamable HTTP und stdio, Key-Authentifizierung, Toolset-Gating, Nutzungs- und Audit-Protokollierung, Antwortbudget	<code>src/mcp/</code> , <code>src/service/</code>
Toolsets	<code>core</code> (11), <code>trace</code> (5), <code>write</code> (12), <code>collab</code> (4), <code>test</code> (7), <code>history</code> (6), <code>review</code> (3), <code>files</code> (5) — zusammen 53	<code>src/mcp/tools/</code>
Komposit-Tools	<code>jama_trace_chain</code> , <code>jama_find_trace_gaps</code> , <code>jama_trace_matrix</code> , <code>jama_compare_baselines</code> , <code>jama_testcycle_summary</code>	<code>src/mcp/tools/</code>
Resources und Prompts	2 Resources, 5 vorgefertigte Abläufe	<code>src/mcp/server.ts</code> , <code>src/mcp/prompts.ts</code>
Admin-Backend	PIN mit Argon2id, IP-Sperre, CSRF, Key- und Verbindungsverwaltung, Auswertung	<code>src/admin/</code>
Admin-Dashboard	9 Seiten: Anmeldung, Übersicht, Keys, Verbindungen, Nutzung, Audit, Aufrufe, Tools, Einstellungen	<code>web/src/pages/</code>
Tests	86 Tests, MCDC-Matrizen für Schutzschichten, Rate-Limiter, Verschlüsselung, Wiederholungslogik	<code>tests/</code>

Nicht umgesetzt

Punkt	Begründung
Contract-Tests gegen eine reale Jama-Instanz	Es lagen keine Zugangsdaten vor. Das ist die wichtigste offene Lücke — Feldnamen, Custom-Field-Suffixe und die Verfügbarkeit der <code>labs</code> -Endpunkte sind ungeprüft
Geteilter Zwischenspeicher (Redis)	Für den Betrieb als einzelner Prozess nicht erforderlich. Das Cache-Interface ist darauf vorbereitet
Aggregat-Tabellen und Partitionierung für <code>usage_events</code>	Erst bei großen Datenmengen nötig; derzeit wird über die Rohdaten aggregiert
Lasttest gegen das Rate-Limit	Sinnvoll erst gegen eine reale Instanz

Punkt	Begründung
Benutzerhandbuch mit Screenshots	Steht aus
Der weitere Ausbau ist in todo.md festgehalten.	

10. Entscheidungen

10.1 Getroffen (2026-08-12)

Thema	Entscheidung	Auswirkung
Betriebsmodell	Zunächst nur lokal per docker-compose , keine Internet-Exposition	nginx und die Port-Trennung werden trotzdem von Anfang an gebaut, damit ein späteres Deployment kein Umbau wird. Die Server-Härtung entfällt vorerst, ist aber vor jeder Exposition ins Internet nachzuholen
Credential-Modell	Beides — pro Person als Standard, Service-Accounts zusätzlich für CI und Automatisierung	<code>api_keys.account_type</code> unterscheidet beide; für Service-Accounts erzwingt die UI Projekt-Allowlist, Ablaufdatum und benannten Inhaber (siehe 3.3)
Umfang v1	Direkt inklusive Write (Phasen 0 bis 6)	Write-Guards sind kein Nachrüstthema, sondern Teil des Grundgerüsts: Toolset-Gating, <code>confirm</code> , <code>dryRun</code> , globaler Read-only-Kill-Switch und Audit-Trail entstehen zusammen mit den ersten Write-Tools

10.2 Noch offen

- Zugang zu einer Test-Instanz (blockierend für die Contract-Tests)** — OAuth-Client-ID/Secret oder Basic-Auth-Konto für eine Sandbox- oder Test-Instanz. Ohne echten Zugang lassen sich Feldnamen, Custom-Field-IDs (`customField$NN`) und die Verfügbarkeit der `labs` -Endpoints nicht gegen ein reales System verifizieren. Die Implementierung selbst ist davon nicht blockiert — sie wird gegen Fixtures getestet.
- Jama-Version der Zielinstanz** — entscheidet über die Toolsets `review` (ab 9.32, labs) und `reports` (ab 8.79, labs). Wird beim ersten Verbindungstest automatisch erkannt (`GET /rest`), sollte aber vorab bekannt sein.
- Lizenzlage** — REST-Zugriff erfordert *Named-Creator*-Lizenzen. Creator-Float-Lizenzen haben keinen API-Zugriff. Das begrenzt den möglichen Nutzerkreis und sollte vor dem Rollout geklärt sein.
- Nutzerkreis mittelfristig** — nur intern oder später auch beim Kunden betrieben. Bei Kundenbetrieb würde Mandantenfähigkeit relevant; das Datenmodell ist dafür bereits vorbereitet (Verbindungen sind mehrfach anlegbar), die UI noch nicht.

Quellen

- [Jama REST API Referenz](#)
- [dev.jamasoftware.com — API](#)
- [Jama API Cookbook](#)
- [REST API Throttle Limits \(429\)](#)
- [REST API and extensibility — Jama Help](#)
- [jamasoftware-ps/py-jama-rest-client](#)
- [jamasoftware-ps/REST-References](#)
- [t-j-thomas/jama-mcp-server](#)
- [Jama Software: Launch des offiziellen MCP-Servers](#)